

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №1

«Організація професійного Python-проєкту та Git»

з дисципліни «Професійний Python»

Варіант №5 — «Каталог автомобілів»

Виконав:

студент групи ФЕП-32

Горин Михайло

Репозиторій: github.com/MykhailoG212/Prof_python

Львів — 2026

2. Номер і тема лабораторної роботи

Лабораторна робота №1. Тема: «Організація професійного Python-проєкту та Git».

3. Мета роботи

Опанувати практичні навички організації професійного Python-проєкту: створення ізольованого віртуального середовища (venv), встановлення пакета в режимі розробки (`pip install -e`), побудову структури проєкту за схемою `src-layout`, конфігурування проєкту через `pyproject.toml`, написання автоматичних тестів за допомогою `pytest` та ведення історії змін у системі контролю версій Git із публікацією репозиторію на GitHub.

4. Номер індивідуального варіанта

Варіант №5 — «Каталог автомобілів». Репозиторій проєкту: https://github.com/MykhailoG212/Prof_python

5. Постановка задачі

Необхідно розробити консольний застосунок «Каталог автомобілів», організований як професійний Python-пакет, та налаштувати для нього інфраструктуру розробки й контролю версій.

5.1. Сутність Car

Поле	Тип	Вимога
brand (марка)	str	непорожній рядок
model	str	непорожній рядок
year	int	рік випуску, перевірка <code>year >= 1886</code>
price	float	ціна в USD, <code>price >= 0</code> , скінченне число
mileage	int	пробіг у км, <code>mileage >= 0</code>
full_name	str (property)	обчислювана властивість: «марка модель»

5.2. Бізнес-логіка (services.py)

- пошук автомобілів за маркою без урахування реєстру;
- фільтрація автомобілів за роком випуску;
- знаходження найдорожчого автомобіля;
- знаходження автомобіля з найменшим пробігом;
- обчислення середньої вартості автомобілів.

5.3. Вимоги до архітектури

- `src-layout`: пакет `src/car_catalog/` з модулями `models.py`, `services.py`, `main.py`, `__init__.py`;
- тести `pytest` у `tests/test_services.py`;
- `pyproject.toml`: збірка через `setuptools >= 70`, консольна команда в `project.scripts`, конфігурація `pytest`;
- файли `.gitignore` та `README.md`;
- віртуальне середовище, встановлення `pip install -e "[test]"`;
- історія Git щонайменше з трьох логічних етапних комітів.

6. Структура проєкту

Проект організовано за схемою `src-layout`: вихідний код пакета винесено в каталог `src/`, тести — в окремий каталог `tests/`, службові та конфігураційні файли — у корені.

```
lab1code/
├── .git/                # репозиторій Git
├── .venv/              # віртуальне середовище (ігнорується Git)
├── src/
│   └── car_catalog/
│       ├── __init__.py # ознака пакета, версія
│       ├── models.py   # датаклас Car з валідацією
│       ├── services.py  # бізнес-логіка каталогу
│       └── main.py      # CLI-меню, точка входу main()
├── tests/
│   ├── test_services.py # тести моделі та сервісів
│   └── test_main.py     # тести інтерактивного введення та меню
├── .gitignore
├── pyproject.toml
└── README.md
```

7. Опис створених модулів

Модуль	Призначення
<code>__init__.py</code>	Позначає каталог <code>car_catalog</code> як Python-пакет і містить рядок версії <code>__version__ = "0.1.0"</code> .
<code>models.py</code>	Містить незмінний датаклас <code>Car</code> (<code>frozen=True</code> , <code>slots=True</code>). Метод <code>__post_init__</code> перевіряє коректність даних: непорожні марка й модель, рік ≥ 1886 , ціна ≥ 0 і скінченна, пробіг ≥ 0 ; при порушенні викидається <code>ValueError</code> . Властивість <code>full_name</code> повертає рядок «марка модель».
<code>services.py</code>	Чисті функції бізнес-логіки: <code>find_by_brand</code> (пошук за маркою через <code>casefold()</code>), <code>filter_by_year</code> , <code>find_most_expensive</code> , <code>find_lowest_mileage</code> (через <code>max/min</code> з <code>default=None</code>), <code>calculate_average_price</code> (для порожнього списку повертає 0.0).
<code>main.py</code>	Консольний інтерфейс: демонстраційні дані <code>get_demo_cars()</code> , табличний вивід <code>print_cars()</code> , меню з 8 пунктів <code>run_menu()</code> , безпечне введення з повторним запитом (<code>_read_non_empty</code> , <code>_read_int</code> , <code>_read_price</code>), створення авто <code>create_car_from_input()</code> та точка входу <code>main()</code> .
<code>tests/test_services.py</code>	Набір тестів <code>pytest</code> із фікстурою <code>sample_cars</code> : перевірка пошуку за маркою, фільтрації за роком, найдорожчого авто, найменшого пробігу, середньої ціни, валідації моделі та поведінки на порожньому каталозі.
<code>tests/test_main.py</code>	Додаткові тести інтерактивного введення: підміна <code>input</code> через <code>monkeypatch</code> , перевірка повторних запитів при некоректних даних і додавання авто через меню.

8. Текст основних фрагментів коду

8.1. Файл src/car_catalog/models.py

```
import math
from dataclasses import dataclass

@dataclass(slots=True, frozen=True)
class Car:
    """Модель даних автомобіля.

    Атрибути:
        brand: Марка автомобіля.
        model: Модель автомобіля.
        year: Рік випуску (>= 1886).
        price: Ціна в USD (>= 0).
        mileage: Пробіг у кілометрах (>= 0).
    """

    brand: str
    model: str
    year: int
    price: float
    mileage: int

    def __post_init__(self) -> None:
        if not self.brand.strip():
            raise ValueError("Марка автомобіля не може бути порожньою")
        if not self.model.strip():
            raise ValueError("Модель автомобіля не може бути порожньою")
        if self.year < 1886:
            raise ValueError(f"Некоректний рік випуску: {self.year}")
        if not math.isfinite(self.price) or self.price < 0:
            raise ValueError("Ціна не може бути від'ємною")
        if self.mileage < 0:
            raise ValueError("Пробіг не може бути від'ємним")

    @property
    def full_name(self) -> str:
        return f"{self.brand} {self.model}"
```

8.2. Файл src/car_catalog/services.py

```
from car_catalog.models import Car

def find_by_brand(cars: list[Car], brand: str) -> list[Car]:
    """Повертає список авто відповідної марки (без урахування реєстру)."""
    normalized_brand = brand.strip().casefold()
    return [car for car in cars if car.brand.casefold() == normalized_brand]

def filter_by_year(cars: list[Car], year: int) -> list[Car]:
    """Повертає список авто за вказаним роком випуску."""
    return [car for car in cars if car.year == year]

def find_most_expensive(cars: list[Car]) -> Car | None:
    """Знаходить автомобіль із найвищою ціною."""
    return max(cars, key=lambda car: car.price, default=None)

def find_lowest_mileage(cars: list[Car]) -> Car | None:
    """Знаходить автомобіль із найменшим пробігом."""
```

```

        return min(cars, key=lambda car: car.mileage, default=None)

def calculate_average_price(cars: list[Car]) -> float:
    """Обчислює середню ціну списку автомобілів."""
    if not cars:
        return 0.0
    return sum(car.price for car in cars) / len(cars)

```

8.3. Файл src/car_catalog/main.py

```

import math

from car_catalog.models import Car
from car_catalog.services import (
    calculate_average_price,
    filter_by_year,
    find_by_brand,
    find_lowest_mileage,
    find_most_expensive,
)

def get_demo_cars() -> list[Car]:
    return [
        Car(brand="Tesla", model="Model Y Performance", year=2023, price=50000.0,
            mileage=12000),
        Car(brand="Toyota", model="Camry", year=2020, price=25000.0, mileage=45000),
        Car(brand="Honda", model="Civic", year=2021, price=22000.0, mileage=30000),
        Car(brand="Toyota", model="Corolla", year=2019, price=18000.0, mileage=60000),
    ]

def print_cars(cars: list[Car]) -> None:
    if not cars:
        print("\n[Список порожній]")
        return

    print("\n" + "=" * 70)
    print(f"{'Марка та Модель':<30} | {'Рік':<6} | {'Ціна, $':<12} | {'Пробіг, км':<10}")
    print("-" * 70)
    for car in cars:
        print(f"{car.full_name:25} | Рік: {car.year:4} | Ціна: ${car.price:8.2f} | Пробіг: {car.mileage:7} км")
    print("=" * 70)

def print_menu() -> None:
    print("\n--- КАТАЛОГ АВТОМОБІЛІВ ---")
    print("1. Показати всі автомобілі")
    print("2. Пошук за маркою")
    print("3. Фільтрація за роком випуску")
    print("4. Знайти найдорожчий автомобіль")
    print("5. Знайти авто з найменшим пробігом")
    print("6. Розрахувати середню ціну")
    print("7. Додати автомобіль")
    print("8. Вийти")

def read_non_empty(prompt: str) -> str:
    while True:
        value = input(prompt).strip()
        if value:
            return value
        print("Поле не може бути порожнім.")

def _read_int(prompt: str, minimum: int, error_message: str) -> int:

```

```

while True:
    try:
        value = int(input(prompt).strip())
    except ValueError:
        print("Помилка: введіть ціле число.")
        continue
    if value < minimum:
        print(error_message)
        continue
    return value

def _read_price() -> float:
    while True:
        try:
            price = float(input("Введіть ціну в USD: ").strip())
        except ValueError:
            print("Помилка: введіть коректне число для ціни.")
            continue
        if not math.isfinite(price) or price < 0:
            print("Ціна має бути невід'ємним скінченним числом.")
            continue
        return price

def create_car_from_input() -> Car:
    brand = _read_non_empty("Введіть марку автомобіля: ")
    model = _read_non_empty("Введіть модель автомобіля: ")
    year = _read_int("Введіть рік випуску: ", 1886, "Рік випуску має бути не раніше 1886.")
    price = _read_price()
    mileage = _read_int("Введіть пробіг у кілометрах: ", 0, "Пробіг не може бути від'ємним.")
    return Car(brand=brand, model=model, year=year, price=price, mileage=mileage)

def run_menu(catalog: list[Car]) -> None:
    while True:
        print_menu()
        choice = input("Оберіть дію: ").strip()

        if choice == "1":
            print_cars(catalog)

        elif choice == "2":
            make = input("Введіть марку для пошуку: ").strip()
            results = find_by_brand(catalog, make)
            print_cars(results)

        elif choice == "3":
            raw_year = input("Введіть рік: ").strip()
            try:
                year = int(raw_year)
                results = filter_by_year(catalog, year)
                print_cars(results)
            except ValueError:
                print("Помилка: введіть коректне число для року.")

        elif choice == "4":
            car = find_most_expensive(catalog)
            if car:
                print(f"\nНайдорожче авто: {car.full_name} — ${car.price:,.2f}")
            else:
                print("\nКаталог порожній.")

        elif choice == "5":
            car = find_lowest_mileage(catalog)
            if car:
                print(f"\nАвто з найменшим пробігом: {car.full_name} — {car.mileage:,} км")
            else:
                print("\nКаталог порожній.")

        elif choice == "6":

```

```

        avg = calculate_average_price(catalog)
        print(f"\nСередня ціна авто в каталозі: ${avg:,.2f}")

    elif choice == "7":
        catalog.append(create_car_from_input())
        print("Автомобіль додано до каталогу.")

    elif choice == "8":
        print("\nРоботу завершено.")
        break
    else:
        print("\nНевідома команда. Спробуйте ще раз.")

def main() -> None:
    catalog = get_demo_cars()
    run_menu(catalog)

if __name__ == "__main__":
    main()

```

8.4. Файл tests/test_services.py

```

import pytest

from car_catalog.models import Car
from car_catalog.services import (
    calculate_average_price,
    filter_by_year,
    find_by_brand,
    find_lowest_mileage,
    find_most_expensive,
)

@pytest.fixture
def sample_cars() -> list[Car]:
    return [
        Car("Tesla", "Model Y", 2023, 50000.0, 20000),
        Car("BMW", "M3", 2021, 70000.0, 30000),
        Car("Tesla", "Model 3", 2021, 35000.0, 45000),
    ]

def test_filter_by_make(sample_cars: list[Car]) -> None:
    result = find_by_brand(sample_cars, "tEsLa ")
    assert len(result) == 2

def test_filter_by_year(sample_cars: list[Car]) -> None:
    result = filter_by_year(sample_cars, 2021)
    assert len(result) == 2

def test_find_most_expensive_car(sample_cars: list[Car]) -> None:
    car = find_most_expensive(sample_cars)
    assert car is not None
    assert car.price == 70000.0

def test_find_lowest_mileage_car(sample_cars: list[Car]) -> None:
    car = find_lowest_mileage(sample_cars)
    assert car is not None
    assert car.mileage == 20000

```

```

def test_calculate_average_price(sample_cars: list[Car]) -> None:
    assert calculate_average_price(sample_cars) == pytest.approx(51666.67, 0.01)

def test_car_validation() -> None:
    with pytest.raises(ValueError):
        Car("Test", "Fail", 1800, 100.0, 10)
    with pytest.raises(ValueError):
        Car("Test", "Fail", 2020, -1.0, 10)
    with pytest.raises(ValueError):
        Car("Test", "Fail", 2020, 100.0, -1)
    with pytest.raises(ValueError):
        Car(" ", "Fail", 2020, 100.0, 1)
    with pytest.raises(ValueError):
        Car("Test", " ", 2020, 100.0, 1)
    with pytest.raises(ValueError):
        Car("Test", "Fail", 2020, float("nan"), 1)

def test_empty_catalog_results() -> None:
    assert find_most_expensive([]) is None
    assert find_lowest_mileage([]) is None
    assert calculate_average_price([]) == 0.0

```

8.5. Файл `src/car_catalog/__init__.py`

```

"""Пакет каталогу автомобілів."""

__version__ = "0.1.0"

```

9. Пояснення файлу `pyproject.toml`

Файл `pyproject.toml` — єдине місце конфігурації проекту (стандарти PEP 517, 518, 621). Повний вміст файлу:

```

[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "car-catalog"
version = "0.1.0"
description = "Car catalog laboratory project"
readme = "README.md"
requires-python = ">=3.11"
authors = [{ name = "Михайло Горин" }]

[project.optional-dependencies]
test = ["pytest>=8.0"]

[project.scripts]
car-catalog = "car_catalog.main:main"

[tool.pytest.ini options]
pythonpath = ["src"]
testpaths = ["tests"]

[tool.setuptools.packages.find]
where = ["src"]

```

Секція	Призначення
[build-system]	Описує, чим збирати проєкт. <code>requires = ["setuptools>=70"]</code> — залежності збірки; <code>build-backend = "setuptools.build_meta"</code> — модуль-бекенд, який рп викликає для встановлення.
[project]	Метадані пакета: ім'я <code>car-catalog</code> , версія, опис, <code>readme</code> , мінімальна версія Python (<code>>=3.11</code>), автор.
[project.optional-dependencies]	Необов'язкові групи залежностей (extras). Група <code>test</code> містить <code>pytest>=8.0</code> і встановлюється командою <code>pip install -e ".[test]"</code> .
[project.scripts]	Описує консольні точки входу. Запис <code>car-catalog = "car_catalog.main:main"</code> створює при встановленні команду <code>car-catalog</code> , що викликає функцію <code>main()</code> .
[tool.pytest.ini_options]	Налаштування <code>pytest</code> : <code>pythonpath = ["src"]</code> додає каталог <code>src</code> до шляху імпорту, <code>testpaths = ["tests"]</code> вказує, де шукати тести.
[tool.setuptools.packages.find]	Вказує <code>setuptools</code> шукати пакети в каталозі <code>src</code> (<code>where = ["src"]</code>), що необхідно для <code>src-layout</code> .

10. Вміст файлу .gitignore

Файл `.gitignore` виключає з-під контролю версій згенеровані та локальні файли. Повний текст файлу:

```
# Byte-compiled / optimized / DLL files
__pycache__ /
*.py[cod]
*$py.class

# C extensions
*.so

# Distribution / packaging
```

```

build/
dist/
*.egg-info/

# Virtual Environments
.venv/
venv/
ENV/

# IDE files
.vscode/
.idea/
*.swp

# Testing caches
.pytest_cache/
.coverage
htmlcov/

```

Пояснення до кожного блоку (причини виключення):

Блок	Чому виключено
<code>__pycache__/,</code> <code>*.py[cod],*\$py.class</code>	Байт-код, який Python генерує автоматично; залежить від версії інтерпретатора й відтворюється при запуску.
<code>*.so</code>	Скомпільовані розширення C — артефакти збирання, платформи-залежні.
<code>build/,dist/,*.egg-info/</code>	Артефакти пакування та метадані editable-встановлення; створюються командами збірки/встановлення.
<code>.venv/,venv/,ENV/</code>	Віртуальне середовище велике, прив'язане до конкретної ОС і шляхів; відновлюється з <code>pyproject.toml</code> .
<code>.vscode/,.idea/,</code> <code>*.swp</code>	Особисті налаштування IDE та тимчасові файли редакторів; не стосуються коду проекту.
<code>.pytest_cache/,</code> <code>.coverage,htmlcov/</code>	Кеші та звіти тестування й покриття; генеруються при кожному запуску.

11. Фрагмент README.md

Нижче наведено ключові розділи файлу `README.md` (вимоги, встановлення, запуск, тестування).

```

# Car Catalog

Навчальний проект з дисципліни «Професійний Python». Варіант 5: каталог автомобілів.

## Вимоги
- Python 3.11+

## Встановлення та розгортання

```

```

1. Створення та активація віртуального середовища:
python -m venv .venv
# Windows:
.venv\Scripts\activate
# Linux / macOS:
source .venv/bin/activate
2. Встановлення пакета в режимі розробки разом із тестовими залежностями:
python -m pip install -e ".[test]"

## Запуск застосунку
Запуск як модуль:
python -m car_catalog.main
Або через CLI скрипт:
car-catalog

## Тестування
python -m pytest

```

12. Демонстрація роботи застосунку

Нижче наведено протокол консольного сеансу: встановлення, запуск CLI, робота меню та результат `pytest`. Вивід отримано під час реального запуску проєкту.

Рисунок 1 — Встановлення пакета в режимі розробки

```

$ python -m venv .venv
$ source .venv/bin/activate      # y Windows: .venv\Scripts\activate
(.venv) $ python -m pip install -e ".[test]"
...
Successfully installed car-catalog-0.1.0 iniconfig packaging pluggy pygments pytest-9.1.1

```

Рисунок 1

Рисунок 2 — Запуск застосунку, пункт 1 «Показати всі автомобілі»

```

(.venv) $ car-catalog

--- КАТАЛОГ АВТОМОБІЛІВ ---
1. Показати всі автомобілі
2. Пошук за маркою
3. Фільтрація за роком випуску
4. Знайти найдорожчий автомобіль
5. Знайти авто з найменшим пробігом
6. Розрахувати середню ціну
7. Додати автомобіль
8. Вийти
Оберіть дію: 1

=====
Марка та Модель | Рік | Ціна, $ | Пробіг, км
=====
Tesla Model Y Performance | Рік: 2023 | Ціна: $50000.00 | Пробіг: 12000 км
Toyota Camry | Рік: 2020 | Ціна: $25000.00 | Пробіг: 45000 км
Honda Civic | Рік: 2021 | Ціна: $22000.00 | Пробіг: 30000 км
Toyota Corolla | Рік: 2019 | Ціна: $18000.00 | Пробіг: 60000 км
=====

```

Рисунок 2

Рисунок 3 — Пошук за маркою (регістр не впливає) та фільтрація за роком

```

Оберіть дію: 2
Введіть марку для пошуку: toyota

```

```

=====
Марка та Модель          | Рік      | Ціна, $      | Пробіг, км
-----
Toyota Camry             | Рік: 2020 | Ціна: $25000.00 | Пробіг: 45000 км
Toyota Corolla           | Рік: 2019 | Ціна: $18000.00 | Пробіг: 60000 км
=====

Оберіть дію: 3
Введіть рік: 2021

=====
Марка та Модель          | Рік      | Ціна, $      | Пробіг, км
-----
Honda Civic              | Рік: 2021 | Ціна: $22000.00 | Пробіг: 30000 км
=====

```

Рисунок 3

Рисунок 4 — Статистика: найдорожче авто, найменший пробіг, середня ціна

```

Оберіть дію: 4

Найдорожче авто: Tesla Model Y Performance — $50,000.00

Оберіть дію: 5

Авто з найменшим пробігом: Tesla Model Y Performance — 12,000 км

Оберіть дію: 6

Середня ціна авто в каталозі: $28,750.00

```

Рисунок 4

Рисунок 5 — Додавання автомобіля та вихід

```

Оберіть дію: 7
Введіть марку автомобіля: BMW
Введіть модель автомобіля: X5
Введіть рік випуску: 2022
Введіть ціну в USD: 45000
Введіть пробіг у кілометрах: 15000
Автомобіль додано до каталогу.

Оберіть дію: 8

Роботу завершено.

```

Рисунок 5

Рисунок 6 — Результат запуску pytest

```

(.venv) $ python -m pytest -v
===== test session starts =====
configfile: pyproject.toml
testpaths: tests
collected 9 items

tests/test_main.py::test create car from input retries invalid values PASSED [ 11%]
tests/test_main.py::test menu adds car to catalog PASSED [ 22%]
tests/test_services.py::test filter by make PASSED [ 33%]
tests/test_services.py::test filter by year PASSED [ 44%]
tests/test_services.py::test find most expensive car PASSED [ 55%]
tests/test_services.py::test find lowest mileage car PASSED [ 66%]
tests/test_services.py::test calculate average price PASSED [ 77%]

```

```
tests/test_services.py::test_car_validation PASSED [ 88%]
tests/test_services.py::test_empty_catalog_results PASSED [100%]

===== 9 passed in 0.03s =====
```

Рисунок 6

13. Результат команди git status

Після індексації та фіксації всіх змін робоче дерево чисте, у staging area немає незафіксованих файлів, а локальна гілка синхронізована з віддаленою:

```
$ git add .
$ git commit -m "Align car catalog with report requirements"
$ git push
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

14. Результат команди git log --oneline

Історія репозиторію містить чотири логічні етапні коміти: структура проєкту, модель і бізнес-логіка, точка входу з документацією, фінальне узгодження з вимогами.

```
$ git log --oneline
f1001cb (HEAD -> main, origin/main) Align car catalog with report requirements
9459dc6 Add application entry point and documentation
917f783 Add car model and business logic
bc477be Create project structure
```

Коміт	Зміст етапу
bc477be	Створення структури проєкту (src-layout, pyproject.toml, .gitignore).
917f783	Додано модель Car та бізнес-логіку (models.py, services.py).
9459dc6	Додано точку входу main.py, README.md та тести.
f1001cb	Фінальне узгодження коду та конфігурації з вимогами до звіту.

15. Відповіді на контрольні запитання

1. Що таке віртуальне середовище і навіщо воно потрібне?

Це ізольований каталог із власним інтерпретатором Python і набором пакетів. Воно запобігає конфліктам версій залежностей між проєктами та не засмічує системний Python.

2. Як створити й активувати venv?

Створення: `python -m venv .venv`. Активація: у Windows `.venv\Scripts\activate`, у Linux/macOS `source .venv/bin/activate`. Вихід — команда `deactivate`.

3. Що робить команда `pip install -e .`?

Встановлює пакет у режимі розробки (editable): у середовищі створюється посилання на вихідний код, тож зміни в коді діють одразу без перевстановлення.

4. Що таке `src-layout` і яка його перевага?

Це схема, за якої код пакета лежить у каталозі `src/`, а не в корені. Вона виключає випадковий імпорт із поточного каталогу, змушує працювати зі встановленим пакетом і чітко відокремлює код від службових файлів.

5. Чим `src-layout` відрізняється від `flat-layout`?

У `flat-layout` пакет лежить у корені поруч із конфігураційними файлами й тестами, тому імпорт може «працювати» без встановлення. У `src-layout` пакет доступний лише після встановлення (або через `pythonpath`).

6. Що таке `pyproject.toml`?

Єдиний декларативний файл конфігурації проєкту: метадані, система збірки, залежності, точки входу та налаштування інструментів (pytest тощо). Визначений стандартами PEP 517/518/621.

7. Яке призначення секції `[build-system]`?

Вказує, які пакети потрібні для збирання (`requires`) та який бекенд його виконує (`build-backend`), щоб `pip` міг зібрати проєкт в ізольованому середовищі.

8. Що означає запис `setuptools>=70`?

Для збирання потрібен пакет `setuptools` версії 70 або новішої; `pip` автоматично встановить відповідну версію в ізольоване збиральне середовище.

9. Для чого потрібна секція `[project.scripts]`?

Вона оголошує консольні команди. Запис `car-catalog = "car_catalog.main:main"` при встановленні створює виконуваний файл `car-catalog`, який викликає функцію `main`.

10. Що таке optional-dependencies (extras)?

Додаткові групи залежностей, які не потрібні для роботи пакета, але корисні, наприклад для тестування. Встановлюються так: `pip install -e ".[test]"`.

11. Що налаштовують опції `pythonpath` і `testpaths` у `pytest`?

`pythonpath` додає вказані каталоги (тут `src`) до шляхів імпорту; `testpaths` задає каталоги, де `pytest` шукає тести за замовчуванням.

12. Яка роль файлу `__init__.py`?

Позначає каталог як пакет, виконується при імпорті пакета та може містити метадані й публічний інтерфейс; у проєкті він зберігає `__version__`.

13. Що таке `dataclass` і навіщо параметри `frozen` та `slots`?

`@dataclass` автоматично генерує `__init__`, `__repr__`, `__eq__`. `frozen=True` робить об'єкт незмінним, `slots=True` зменшує витрати пам'яті та забороняє довільні атрибути.

14. Що робить метод `__post_init__`?

Викликається одразу після згенерованого `__init__` і використовується для додаткової валідації або обчислення полів. У `Car` він перевіряє діапазони значень.

15. Чому валідацію варто розміщувати в моделі?

Тоді некоректний об'єкт неможливо створити в жодному місці програми: інваріанти даних гарантуються централізовано, а не повторюються в кожному клієнті.

16. Що таке `@property` і де вона використана?

Декоратор, що дає змогу викликати метод як атрибут. У `Car` властивість `full_name` обчислює рядок «марка модель» без зберігання окремого поля.

17. Чому для пошуку за маркою використано `casefold()`?

`casefold()` виконує агресивне зведення до нижнього регістру, коректніше за `lower()` для різних алфавітів, і забезпечує порівняння без урахування регістру.

18. Навіщо в `max()` та `min()` параметр `default=None`?

Без нього виклик на порожньому списку викличе `ValueError`. З `default=None` функція повертає `None`, а виклична сторона обробляє порожній каталог.

19. Чому середня ціна порожнього каталогу дорівнює 0.0?

Щоб уникнути ділення на нуль. Це явно визначена поведінка, перевірена тестом `test_empty_catalog_results`.

20. Що таке анотації типів і як читати запис `Car | None`?

Підказки типів документують очікувані типи й дозволяють статичним аналізаторам знаходити помилки. `Car | None` означає «об'єкт `Car` або `None`» (синтаксис PEP 604).

21. Для чого використовується конструкція `if __name__ == "__main__":`?

Код усередині виконується лише при прямому запуску модуля, але не при його імпорті; завдяки цьому модуль можна безпечно підключати в тестах.

22. Чим відрізняються запуски `python -m car_catalog.main` та `car-catalog`?

Перший запускає модуль через інтерпретатор і не потребує консольного скрипта; другий — команда, створена з `[project.scripts]` під час встановлення пакета.

23. Що таке фікстура `pytest`?

Функція з декоратором `@pytest.fixture`, яка готує дані або стан для тестів; `pytest` передає її результат у тест за іменем параметра (`sample_cars`).

24. Як `pytest` знаходить тести?

У каталогах `testpaths` він збирає файли `test_*.py` (або `*_test.py`), у них — функції та методи з префіксом `test`.

25. Як перевірити, що код викидає виняток?

Через контекстний менеджер `with pytest.raises(ValueError):` — тест пройде, лише якщо всередині блоку виникне вказаний виняток.

26. Для чого в тестах використано `monkeypatch`?

Фікстура `monkeypatch` тимчасово підмінює об'єкти, наприклад `builtins.input`, щоб імітувати введення користувача без реального діалогу.

27. Що таке `.gitignore` і що не слід комітити?

Файл зі списком шаблонів, які Git ігнорує. Не комітять згенероване й локальне: `__pycache__`, `.venv`, `*.egg-info`, кеші, налаштування IDE, секрети.

28. Які три області має Git і як вони пов'язані з командами?

Робоча директорія → `git add` → індекс (staging area) → `git commit` → локальний репозиторій → `git push` → віддалений репозиторій. Стан показує `git status`.

29. Що таке логічний (атомарний) коміт і яким має бути повідомлення?

Коміт, що містить одну завершену зміну. Повідомлення стисле, у наказовій формі й відповідає на питання «що зроблено», наприклад «Add car model and business logic».

30. Як підключити локальний репозиторій до GitHub?

`git remote add origin <URL>`, потім `git push -u origin main`. Опція `-u` пов'язує локальну гілку з віддаленою, і далі достатньо `git push`.

16. Висновок

У ході лабораторної роботи розроблено консольний застосунок «Каталог автомобілів» (варіант №5) і організовано його як професійний Python-проект. Створено віртуальне середовище `venv` та виконано встановлення пакета в режимі розробки командою `pip install -e ".[test]"`, що забезпечує ізолюваність залежностей і миттєве застосування змін у коді.

Застосовано схему `src-layout`, яка відокремлює код пакета від тестів і службових файлів та запобігає випадковим імпортам. Налаштовано `pyproject.toml` із системою збірки `setuptools >= 70`, консольною командою `car-catalog` у `project.scripts`, групою тестових залежностей та конфігурацією `pytest`.

Бізнес-логіку покрито автоматичними тестами `pytest` (9 тестів, усі успішні), які перевіряють валідацію моделі, пошук, фільтрацію, статистичні функції та інтерактивне введення. Для контролю версій використано Git: підготовлено файл `.gitignore`, створено чотири логічні етапні коміти, а репозиторій опубліковано на GitHub. Отримані навички є основою для розробки та супроводу подальших навчальних і практичних Python-проектів.